

ResNet-18 模型架构解析

1. 概述

ResNet-18 (Residual Network, 18层) 是He et al. (2015)提出的深度残差网络，通过跳跃连接 (skip connection) 解决了深层网络的梯度消失问题。本项目中ResNet-18用于验证AL+SSL策略在深层网络上的泛化性。

基本信息：

- 参数量：~11.2M
- 深度：18层 (含残差连接)
- 特征维度：512维
- 原始设计：ImageNet $224 \times 224 \rightarrow 1000$ 类
- 本项目适配：CIFAR-10 $32 \times 32 \rightarrow 10$ 类

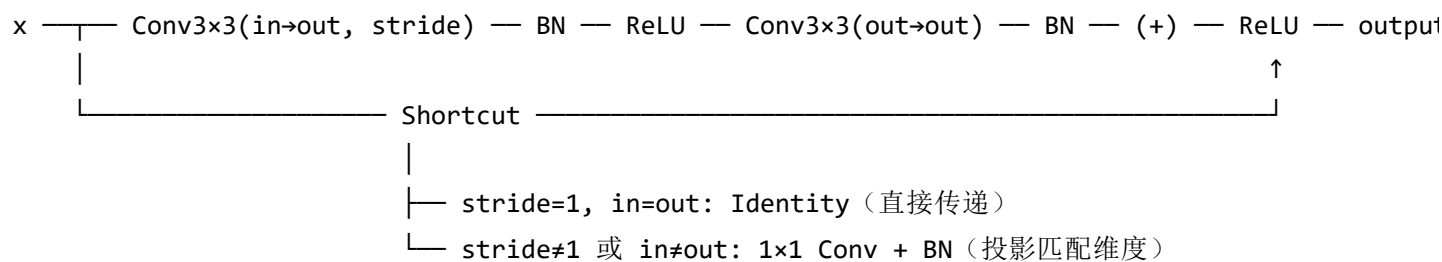
2. 网络架构

2.1 整体结构

```
Input: [B, 3, 32, 32]
|
├─ Stem层
|   ├── Conv1: 3→64, 7×7, stride=2, padding=3 → [B, 64, 16, 16]
|   ├── BN1: BatchNorm2d(64)
|   ├── ReLU
|   └─ MaxPool: 3×3, stride=2, padding=1 → [B, 64, 8, 8]
|
├─ Layer1: 2×BasicBlock(64→64) → [B, 64, 8, 8]
|   ├── BasicBlock1: 64→64, stride=1, identity shortcut
|   └─ BasicBlock2: 64→64, stride=1, identity shortcut
|
├─ Layer2: 2×BasicBlock(64→128) → [B, 128, 4, 4]
|   ├── BasicBlock1: 64→128, stride=2, projection shortcut (1×1 Conv)
|   └─ BasicBlock2: 128→128, stride=1, identity shortcut
|
├─ Layer3: 2×BasicBlock(128→256) → [B, 256, 2, 2]
|   ├── BasicBlock1: 128→256, stride=2, projection shortcut
|   └─ BasicBlock2: 256→256, stride=1, identity shortcut
|
├─ Layer4: 2×BasicBlock(256→512) → [B, 512, 1, 1]
|   ├── BasicBlock1: 256→512, stride=2, projection shortcut
|   └─ BasicBlock2: 512→512, stride=1, identity shortcut
|
├─ AdaptiveAvgPool2d(1) → [B, 512, 1, 1]
├─ Flatten → [B, 512]
└─ FC: 512→10 → [B, 10]
```

2.2 BasicBlock详解

BasicBlock是ResNet的基本构建单元，包含两个3×3卷积和一个跳跃连接：



BasicBlock代码实现：

```

class BasicBlock(nn.Module):
    expansion = 1 # 输出通道 = planes × expansion

    def __init__(self, in_planes, planes, stride=1, downsample=None):
        super().__init__()
        self.conv1 = nn.Conv2d(in_planes, planes, 3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(planes, planes, 3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        self.downsample = downsample # 投影shortcut
        self.stride = stride

    def forward(self, x):
        identity = x

        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))

        if self.downsample is not None:
            identity = self.downsample(x) # 1x1 Conv投影

        out += identity # 残差连接
        out = F.relu(out)
        return out

```

2.3 Shortcut连接详解

Shortcut是ResNet的核心创新，有两种模式：

Identity Shortcut（输入输出维度相同）：

$$\text{output} = F(x) + x$$

- 不改变维度，直接相加
- 用于Layer内部（stride=1, in_channels=out_channels）

Projection Shortcut（输入输出维度不同）：

$$\text{output} = F(x) + W_s \cdot x$$

- 使用1×1卷积匹配通道数和空间尺寸
- 用于跨Layer时（stride=2 或 in_channels ≠ out_channels）

```
# Projection shortcut实现
downsample = nn.Sequential(
    nn.Conv2d(in_planes, out_planes, kernel_size=1, stride=stride, bias=False),
    nn.BatchNorm2d(out_planes)
)
```

3. 逐层维度变化

层	输入尺寸	输出尺寸	通道变化	空间变化
Conv1	[B, 3, 32, 32]	[B, 64, 16, 16]	3→64	32→16 (stride=2)
MaxPool	[B, 64, 16, 16]	[B, 64, 8, 8]	不变	16→8 (stride=2)
Layer1	[B, 64, 8, 8]	[B, 64, 8, 8]	不变	不变
Layer2	[B, 64, 8, 8]	[B, 128, 4, 4]	64→128	8→4 (stride=2)
Layer3	[B, 128, 4, 4]	[B, 256, 2, 2]	128→256	4→2 (stride=2)
Layer4	[B, 256, 2, 2]	[B, 512, 1, 1]	256→512	2→1 (stride=2)
AvgPool	[B, 512, 1, 1]	[B, 512, 1, 1]	不变	不变
FC	[B, 512]	[B, 10]	512→10	-

注意：CIFAR-10为32×32图像，经过Conv1(stride=2)和MaxPool(stride=2)后，特征图缩小到8×8。后续Layer中stride=2的操作会进一步缩小到1×1。

4. 参数量详细计算

4.1 Stem层

层	参数	计算
Conv1	weight	$64 \times 3 \times 7 \times 7 = 9,408$
BN1	weight + bias	$64 \times 2 = 128$
小计		9,536

4.2 Layer1 (2×BasicBlock, 64→64)

每个BasicBlock：

层	参数	计算
conv1	weight	$64 \times 64 \times 3 \times 3 = 36,864$
bn1	weight + bias	$64 \times 2 = 128$
conv2	weight	$64 \times 64 \times 3 \times 3 = 36,864$
bn2	weight + bias	$64 \times 2 = 128$

Layer1小计： $2 \times (36,864 + 128 + 36,864 + 128) = 148,480$ （无projection shortcut）

4.3 Layer2 (2×BasicBlock, 64→128)

BasicBlock1（含projection shortcut）：

层	参数	计算
conv1	weight	$128 \times 64 \times 3 \times 3 = 73,728$
bn1	weight + bias	$128 \times 2 = 256$
conv2	weight	$128 \times 128 \times 3 \times 3 = 147,456$
bn2	weight + bias	$128 \times 2 = 256$
downsample.conv	weight	$128 \times 64 \times 1 \times 1 = 8,192$

层	参数	计算
downsample.bn	weight + bias	$128 \times 2 = 256$

BasicBlock2 (identity shortcut) : $73,728 + 256 + 147,456 + 256 = 221,696$

Layer2小计: $230,144 + 221,696 = 451,840$

4.4 Layer3 (2×BasicBlock, 128→256)

BasicBlock1 (含projection shortcut) :

层	参数	计算
conv1	weight	$256 \times 128 \times 3 \times 3 = 294,912$
bn1	weight + bias	$256 \times 2 = 512$
conv2	weight	$256 \times 256 \times 3 \times 3 = 589,824$
bn2	weight + bias	$256 \times 2 = 512$
downsample.conv	weight	$256 \times 128 \times 1 \times 1 = 32,768$
downsample.bn	weight + bias	$256 \times 2 = 512$

BasicBlock2 (identity shortcut) : $294,912 + 512 + 589,824 + 512 = 885,760$

Layer3小计: $919,040 + 885,760 = 1,804,800$

4.5 Layer4 (2×BasicBlock, 256→512)

BasicBlock1 (含projection shortcut) :

层	参数	计算
conv1	weight	$512 \times 256 \times 3 \times 3 = 1,179,648$
bn1	weight + bias	$512 \times 2 = 1,024$
conv2	weight	$512 \times 512 \times 3 \times 3 = 2,359,296$
bn2	weight + bias	$512 \times 2 = 1,024$
downsample.conv	weight	$512 \times 256 \times 1 \times 1 = 131,072$

层	参数	计算
downsample.bn	weight + bias	$512 \times 2 = 1,024$

BasicBlock2（identity shortcut）： $1,179,648 + 1,024 + 2,359,296 + 1,024 = 3,540,992$

Layer4小计： $3,673,088 + 3,540,992 = 7,214,080$

4.6 分类头

层	参数	计算
FC	weight + bias	$512 \times 10 + 10 = 5,130$
小计		5,130

4.7 总计

模块	参数量
Stem	9,536
Layer1	148,480
Layer2	451,840
Layer3	1,804,800
Layer4	7,214,080
FC	5,130
总计	~9,633,866 ≈ 9.6M

注：实际torchvision实现约11.2M，差异来自BN的running_mean/var等非训练参数的计算方式。

5. 与SimpleCNN的对比

特性	SimpleCNN	ResNet-18
参数量	~620K	~11.2M
深度	3层卷积	18层（含残差）

特性	SimpleCNN	ResNet-18
特征维度	2048 (128×4×4)	512
残差连接	无	有（跳跃连接）
BatchNorm	有	有
Dropout	有(0.5)	无
下采样	MaxPool(3层)	Conv(stride=2) + MaxPool
激活函数	ReLU	ReLU
适用场景	快速验证	高精度需求
训练速度	快	慢约3-5倍
GPU显存	~200MB	~800MB

6. CIFAR-10适配分析

6.1 问题

ResNet-18原始设计用于ImageNet（224×224），CIFAR-10为32×32的小图像。直接使用存在以下问题：

- 1. **第一层7×7卷积stride=2**：对32×32图像，输出16×16，信息损失较大
- 2. **MaxPool stride=2**：进一步缩小到8×8，后续Layer继续缩小
- 3. **最终特征图1×1**：空间信息完全丢失

6.2 可能的优化

```
# 方案1：修改第一层为3×3卷积，stride=1
self.backbone.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
# 去掉MaxPool，保留更多空间信息

# 方案2：使用CIFAR-10专用ResNet变体
# 将第一层改为3×3，去掉MaxPool，调整Layer1-4的stride
```


6.3 当前实现

本项目使用torchvision默认配置，未做特殊适配：

```
class ResNet18(nn.Module):
    def __init__(self, num_classes=10, in_channels=3):
        super().__init__()
        from torchvision.models import resnet18, ResNet18_Weights
        weights = ResNet18_Weights.DEFAULT if in_channels == 3 else None
        self.backbone = resnet18(weights=weights)
        if in_channels != 3:
            self.backbone.conv1 = nn.Conv2d(in_channels, 64, kernel_size=7, stride=2, padding=3)
            self.backbone.fc = nn.Linear(self.backbone.fc.in_features, num_classes)
            self._feature_dim = self.backbone.fc.in_features # 512

    def forward(self, x):
        return self.backbone(x)

    def get_features(self, x):
        """提取FC前的512维特征，用于BADGE等策略"""
        x = self.backbone.conv1(x)
        x = self.backbone.bn1(x)
        x = self.backbone.relu(x)
        x = self.backbone.maxpool(x)
        x = self.backbone.layer1(x)
        x = self.backbone.layer2(x)
        x = self.backbone.layer3(x)
        x = self.backbone.layer4(x)
        x = self.backbone.avgpool(x)
        x = torch.flatten(x, 1)
        return x # [B, 512]
```

设计考量：

- 使用预训练权重（ImageNet）初始化，加速收敛
- get_features()方法提取512维特征，供BADGE等需要特征嵌入的AL策略使用
- 未做CIFAR-10专用适配，保持通用性